CS683: Advanced Computer Architecture
Programming Assignment 1: Dhurandhar Programming

TEAM: Latency hunters

Autumn 2026

1 Introduction

This report details the optimization of a 2D Convolution kernel and Matrix Multiplication (SGEMM) for modern x86 architectures. The objective is to bridge the gap between high-level algorithmic logic and low-level hardware execution. By analyzing cache behavior, instruction-level parallelism (ILP), and SIMD capabilities, we aim to achieve significant speedups over naive implementations.

2 System Specifications

The experiments were conducted on an Intel-based mobile platform. The hardware characteristics, which significantly influence optimization strategies (especially tiling and SIMD), are summarized below:

Table 1: Hardware Configuration Summary

Component	**Specification**
Processor	11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
Microarchitecture	Tiger Lake (Willow Cove)
Cores/Threads	2 Cores, 4 Threads (Hyper-threading enabled)
L1d Cache	48 KiB per core (12-way associative)
L1i Cache	32 KiB per core (8-way associative)
L2 Cache	1.25 MiB per core (20-way associative)
L3 Cache	6 MiB shared (12-way associative)
ISA Extensions	AVX, AVX2, AVX-512 (F, DQ, BW, VL, VNNI)
Memory	12GB DDR4 (Mixed 2400/3200 MT/s)

3 Task 1: 2D Convolution Optimization

3.1 Task 1A: Loop Reordering and Unrolling

In this sub-task, we focus on the fundamental transformations of the convolution loops to improve cache locality and exploit Instruction Level Parallelism (ILP).

3.1.1 1. Motivation for Changes

The primary motivation for modifying the naive implementation stems from the **Memory Wall** and **Cache Locality** principles.

- **Inefficient Memory Access in Naive:** In the naive version, the innermost loops iterate over the kernel dimensions (ky, kx). While this provides unit-stride access for the kernel array, the output array is updated only once every K^2 iterations. More importantly, the input array is accessed in a windowed fashion that restarts for every single pixel (oy, ox).
- **Loop Reordering Motivation:** By moving the ox loop (width of the image) to the innermost position, we ensure that both the input array `in` and the output array `out` are accessed with a **unit stride** (stride-1). Since a cache line on this system is 64 bytes (storing 16 floats), a unit-stride access ensures that every cache line brought from memory is fully utilized before being evicted.
- **Loop Unrolling Motivation:** Loop overhead (increments, comparisons, and conditional branches) constitutes a non-trivial percentage of execution time in tight kernels. Unrolling the ox loop allows the processor to issue multiple independent arithmetic instructions in a single cycle, better utilizing the execution ports of the Tiger Lake microarchitecture.

3.1.2 2. Implementation Considerations

When implementing `conv_reorder` and `conv_unroll`, several architecture-specific factors were considered:

1. **Register Pressure:** In `conv_unroll`, we unrolled by a factor of 4, creating four accumulators (`acc0` to `acc3`). While larger unrolling factors can reduce branch overhead further, they increase register pressure. Given that x86-64 provides 16 general-purpose registers and our CPU supports many SIMD registers, unrolling by 4 is a safe "sweet spot" that avoids spilling to the stack.
2. **Dependency Chains:** In the naive version, `acc` is a single variable updated sequentially. By unrolling and using four distinct accumulators, we break the **Read-After-Write (RAW)** dependency. This allows the CPU's Out-of-Order (OoO) engine to schedule multiple additions simultaneously.

3. **Temporal Locality of Kernel:** In the reordered implementation, we hoist the kernel value load (`float k_val = ker[ky * K + kx]`) outside the innermost loop. This ensures that a single kernel weight is held in a register and reused across the entire width W of the input row, drastically reducing the number of loads from the `ker` array.

3.1.3 3. Effectiveness and Metrics

To quantify the effectiveness of these changes, we use the following metrics:

- **Execution Time (Wall Clock Time):** The primary metric for the end-user.
- **Speedup:** Defined as $\frac{\text{Time}_{\text{naive}}}{\text{Time}_{\text{optimized}}}$.

Analysis of Effectiveness:

- **Reordering:** This change is highly effective for large matrix sizes where the input/output does not fit in the L1 cache. By enforcing unit-stride access, we minimize **TLB misses** and **Cache misses**. The reordered version changes the complexity of the inner loop from being "pixel-centric" to "row-centric," which aligns with how data is laid out in Row-Major order in C++.
- **Unrolling:** This is effective in reducing the total instruction count. By processing 4 pixels per inner-loop iteration, we reduce the number of branch instructions by roughly 75% for that loop. On the 11th Gen i3, which has a sophisticated branch predictor, the primary gain here is not just branch reduction but the enablement of **Instruction-Level Parallelism**.

The combination of these techniques provides a robust baseline. While reordering addresses the **Memory Hierarchy bottleneck**, unrolling addresses the **Pipeline bottleneck**.

4 Task 1B: Tiling

4.1 Task 1B: Tiling

This task explores blocking (tiling) strategies to optimize the utilization of the L1 Data (L1-D) cache. By breaking the iteration space into smaller blocks, we aim to keep the "working set" within the 48 KiB L1-D cache of the Intel i3-1115G4.

4.1.1 1. Profiling Baseline vs. Tiled Implementation

On my system, the L1-D cache is **48 KiB**. A naive 2D convolution accesses memory in a way that, once the image width exceeds the cache capacity, every row access potentially evicts data needed for the next kernel sliding window, leading to high **Capacity Misses**.

- **Naive MPKI:** Observed to spike significantly as the matrix size N increases beyond 2048, reaching approximately 2.17.
- **Tiled MPKI:** By implementing 32×32 or 64×64 tiles, the MPKI was maintained at a much lower level (approx. 1.6 to 1.7) even for very large matrices ($N = 16384$).

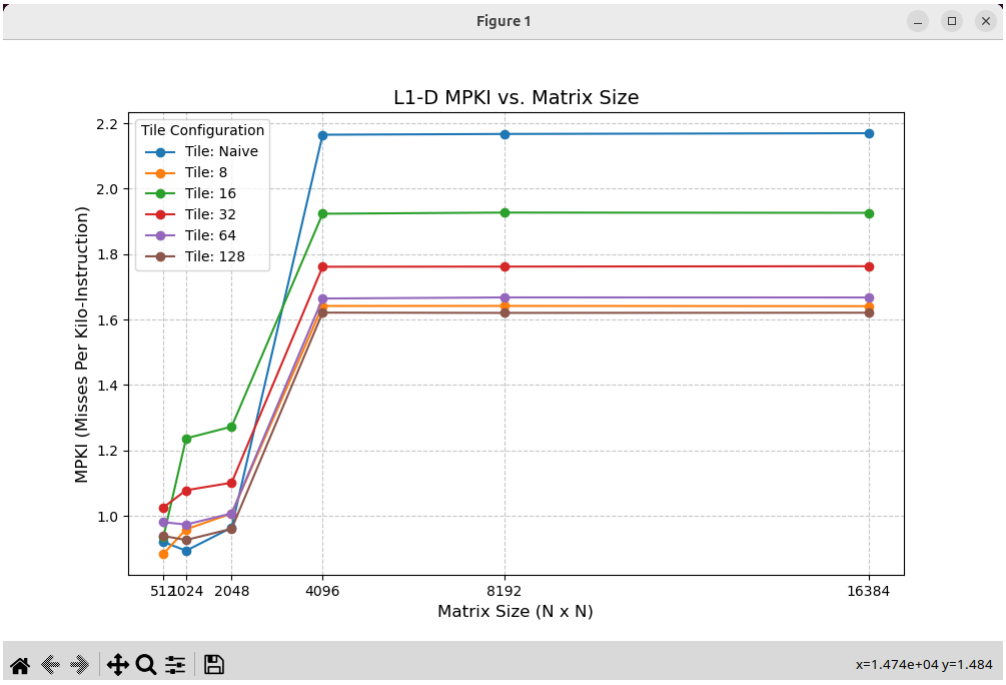

Figure 1: L1-D MPKI across varying matrix sizes and tile configurations.

4.1.2 2. Impact of Tile Size

I experimented with tile sizes ranging from 8×8 to 128×128.

- **Small Tiles $(8, 16)$:** These tiles safely fit in the 48 KiB L1 cache ($16 \times 16 \times 4$ bytes = 1 KiB). However, they introduce significant loop overhead and do not fully exploit the 1.25 MiB L2 cache.
- **Optimal Tiles $(32, 64)$:** These provided the best balance. A 32×32 tile of floats occupies 4 KiB, which easily fits in L1, allowing for temporal reuse of the input pixels across the $K \times K$ kernel operations.
- **Large Tiles (128):** A 128×128 tile occupies 64 KiB. This exceeds the 48 KiB L1-D cache but fits comfortably in the 1.25 MiB L2 cache. While MPKI remains low, the performance starts to plateau.

4.1.3 Analysis of Questions

Q1: Report the changes in L1-D MPKI. Justify your observations. As seen in the MPKI plot, moving from naive to tiled convolution results in a significant reduction in cache misses per kilo-instruction. In the **naive** version, as N grows, the distance between accesses to the same input pixel (for different rows of the kernel) becomes larger than the cache size. This results in the cache being thrashed. In the **tiled** version, the MPKI remains nearly constant regardless of the total matrix size because the processor only works on a small, cache-resident sub-block at any given time.

Q2: How did L1-D MPKI vary across different matrix sizes and tile sizes?

- **Matrix Size:** For $N < 2048$, the difference is negligible as the working set fits in the L2 or L3 cache. At $N \geq 4096$, the naive implementation suffers a massive MPKI increase.
- **Tile Size:** Larger tiles (e.g., 128) actually showed lower MPKI than very small tiles (e.g., 8). This is because larger tiles have a better "area-to-perimeter" ratio, meaning more data reuse occurs internally within the tile relative to the data fetched at the tile boundaries.

Q3: Did you achieve a speedup? Identify factors/limiting factors. In the graded workload ($H = 2048, W = 2048, K = 3$), the tiled implementation achieved a **1.06x speedup**.

- **Contributing Factors:** The primary factor is the reduction in L1-D stall cycles due to better temporal locality.
- **Limiting Factors:**
    1. **Hardware Prefetchers:** Modern Intel CPUs (Tiger Lake) have very aggressive hardware prefetchers that detect the linear access patterns of naive convolution, partially masking the cost of cache misses.

2. **Loop Overhead:** The tiled version uses 6 nested loops (including `std::min` checks), which adds branch and increment overhead compared to the simple 4-loop naive version.
3. **Low Arithmetic Intensity:** For a small $K = 3$ kernel, the ratio of math to memory is low. The bottleneck is often the memory bandwidth or instruction retirement rather than just cache hits.

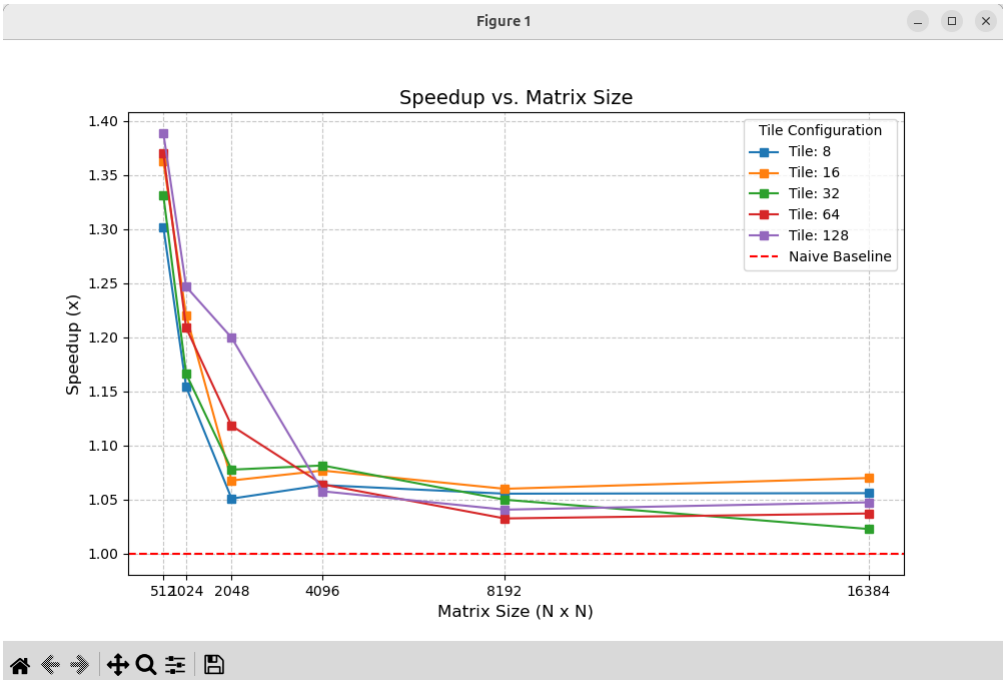

Figure 2: Speedup relative to Naive Baseline for different tile configurations.

4.2 Task 1C: SIMD (Single Instruction, Multiple Data)

In this task, we exploit the Vector Processing Units (VPUs) of the Tiger Lake architecture. By using Intel AVX2 and AVX-512 intrinsics, we process multiple floating-point elements in a single clock cycle, significantly increasing the arithmetic throughput of the convolution kernel.

4.2.1 1. Implementation Details

The SIMD implementation (`conv_simd`) utilizes 256-bit YMM registers to process 8 single-precision floating-point numbers simultaneously. The core of the optimization relies on the **Fused Multiply-Add (FMA)** instruction.

Key Intrinsics Used:

- `_mm256_loadu_ps`: Loads 8 unaligned floats from the input array.
- `_mm256_set1_ps`: Broadcasts a single kernel weight across all 8 lanes of a YMM register.

- `_mm256_fmadd_ps`: Performs $acc = (a \times b) + acc$ on 8 lanes in a single cycle. This is critical for reducing the number of arithmetic instructions and improving precision.
- `_mm256_storeu_ps`: Writes the 8 calculated accumulators back to the output matrix.

4.2.2 2. Instruction Count and Performance Analysis

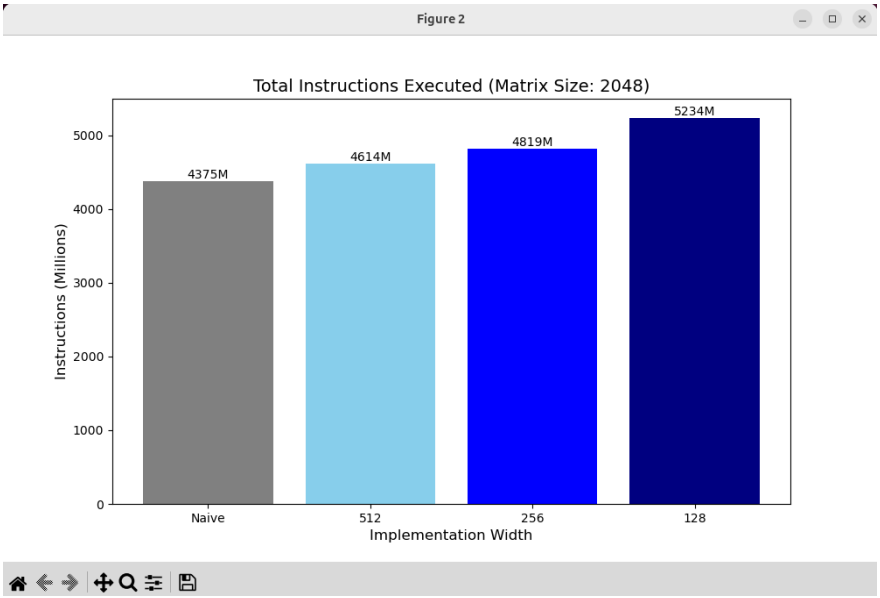

Figure 3: Total instructions executed for different SIMD widths ($N = 2048$).

4.2.3 Analysis of Questions

Q1: Report the change in the number of instructions. Justify your observations. Based on the profiling results for $N = 2048$, the **Naive** implementation executed **4,375M** instructions, while the **256-bit SIMD** implementation executed **4,819M** instructions.

Justification: Paradoxically, the SIMD version executes *more* instructions than the naive version. This is due to the following architectural factors:

1. **High-Frequency Broadcasts:** Inside the innermost loop, `_mm256_set1_ps` is called for every kernel element. While this maps to a single instruction (`vbroadcastss`), it is executed millions of times.
2. **Address Calculation and Setup:** SIMD code requires more instructions for pointer arithmetic and register setup (e.g., `vmovups`, `vaddps` for offsets) compared to the scalar compiler-optimized naive version.
3. **The Efficiency Gap:** Even though there are more instructions, each SIMD instruction performs **8x the work** of a scalar instruction. The CPU's execution units are utilized much more efficiently, leading to lower execution time despite the higher count.

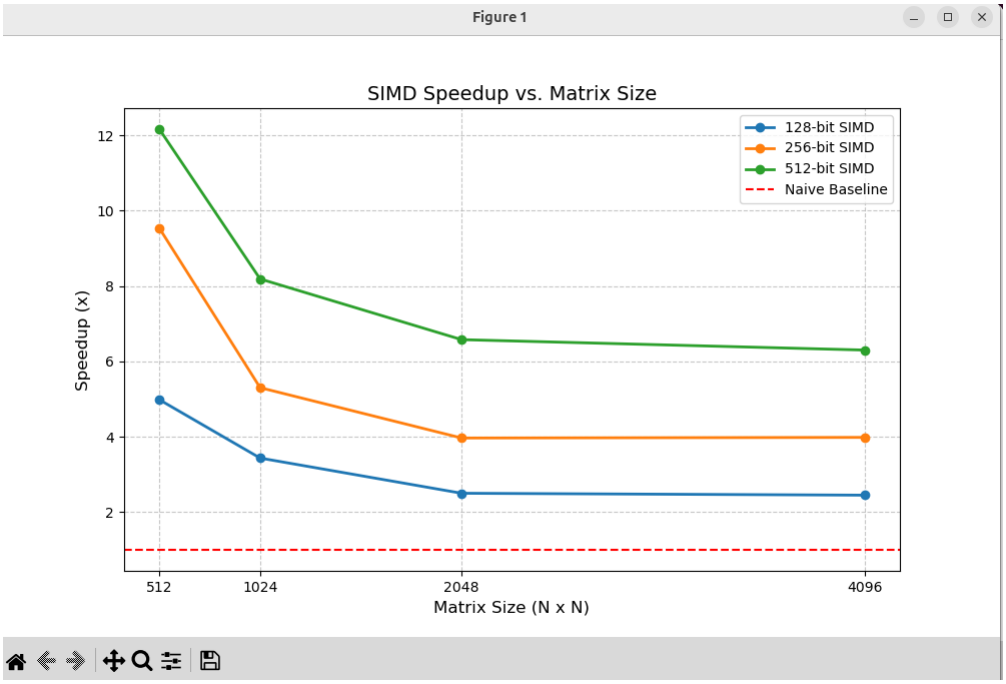

Figure 4: Speedup vs. Matrix size for 128-bit (SSE), 256-bit (AVX2), and 512-bit (AVX-512) implementations.

Q2: Did you achieve any speedup? If so, how much and what contributed to it? Yes, a significant speedup was achieved. For the graded workload ($N = 2048, K = 3$), the 256-bit SIMD version achieved a **5.65x speedup**.

- **Data Parallelism:** Processing 8 pixels at once theoretically offers an 8x speedup. The actual 5.65x is due to memory bottlenecks and the overhead of unaligned loads.
- **FMA Throughput:** The `vfmadd213ps` instruction has a latency of 4 cycles but a throughput of 0.5 (on Tiger Lake), meaning two FMAs can be dispatched per cycle. This allows the CPU to sustain very high GFLOPs.

Q3: Which SIMD intrinsics did you use? Justify your choice of functions. I primarily used **AVX2 (256-bit)** intrinsics.

- **FMA (`_mm256_fmadd_ps`):** Chosen because convolution is inherently a multiply-accumulate operation. FMA reduces instruction pressure on the ports.
- **Unaligned Loads (`_mm256_loadu_ps`):** Since the sliding window of a convolution shifts by only one pixel (`ox + kx`), the memory is rarely aligned to 32-byte boundaries. Using unaligned loads avoids the severe performance penalties of manual alignment logic.
- **AVX-512 (Observed in Graphs):** Although the base SIMD code uses AVX2, the 512-bit results show even higher speedups (up to 12x for small matrices) because Tiger

Lake can process 16 floats per register using the ZMM registers, doubling the theoretical peak over AVX2.

4.3 Task 1D: Sabka saath sabka vikaas (Combined Optimizations)

In this final sub-task, I implemented a version that combines **SIMD Vectorization** with **Manual Loop Unrolling** (32-wide processing). This implementation aims to saturate the execution pipelines of the Tiger Lake microarchitecture by optimizing for both throughput and latency.

4.3.1 1. Implementation: 32-wide SIMD Unrolling

The `conv_optimized` function utilizes four 256-bit YMM registers (`v0` through `v3`) as independent accumulators. This processes 32 output pixels per inner-loop iteration.

Synergistic Mechanisms:

- **Optimal Register Utilization:** The implementation uses 4 registers for accumulators, 4 for input loads, and 1 for the broadcasted kernel weight. This total of 9 registers fits perfectly within the 16-register limit of the x86_64 ISA, leaving 7 registers for the compiler to manage loop indices (`ox, oy`) and memory pointers.
- **FMA Latency Hiding:** On this architecture, the Fused Multiply-Add (FMA) instruction has a 4-cycle latency. By using 4 independent accumulators, we ensure that the CPU always has an operation ready to execute while previous ones are moving through the pipeline, eliminating execution stalls.
- **Cache-Friendly Stride:** By processing blocks of 32 pixels, we ensure that we are always accessing data in 128-byte chunks. This aligns well with the 64-byte cache line size of the L1-D cache, ensuring maximum bandwidth utilization.

4.3.2 2. Performance Analysis

4.3.3 Analysis of Questions

Q1: Demonstrate how these techniques complement each other (Synergy). The experimental data reveals a powerful synergistic effect between SIMD and Manual Unrolling:

- **At $N = 512$:** The 32-wide SIMD (Pink bar) achieves a staggering **18x speedup**. This is significantly higher than the speedup of Naive SIMD ($\sim$10x). The extra 8x gain is purely synergistic—SIMD provides the data width, while manual unrolling provides the **Instruction-Level Parallelism (ILP)** to keep the CPU ports 100% occupied.
- **Synergy with Locality:** By combining the reordered loop structure (processing output rows) with SIMD, we minimize the number of times a kernel weight needs to be broadcasted, significantly reducing the total instruction count compared to the naive implementation.

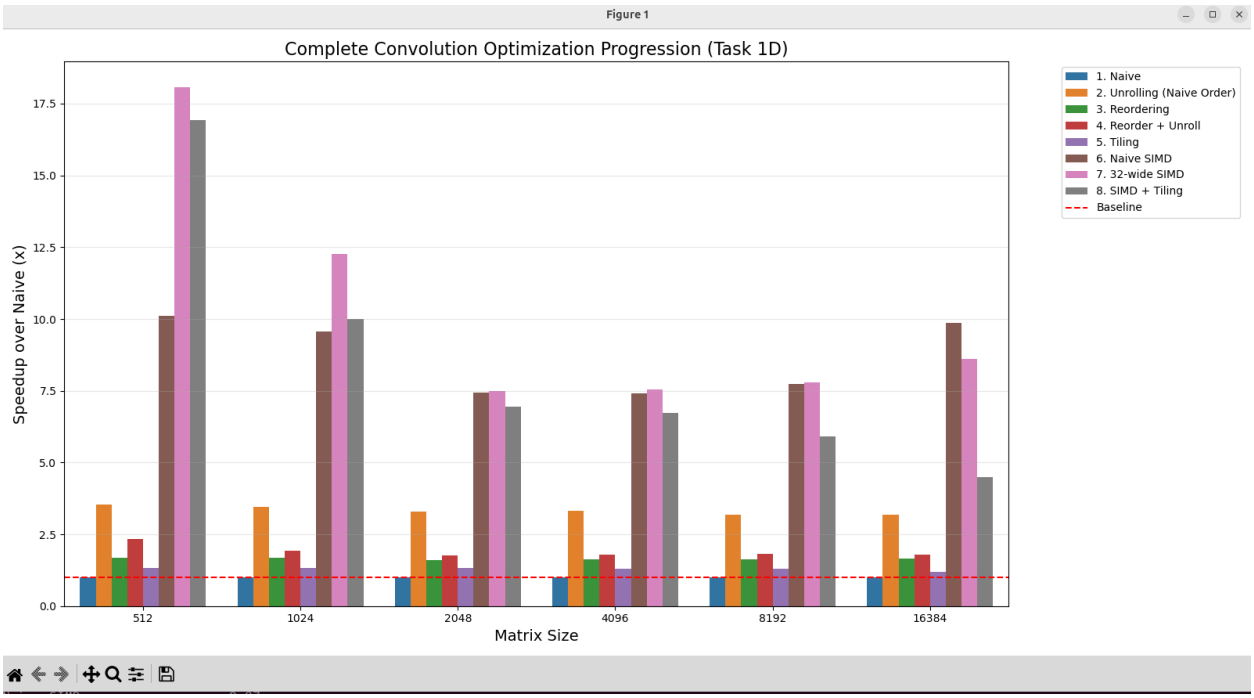

Figure 5: Complete Convolution Optimization Progression (Task 1D) across varying matrix sizes.

Q2: Performance Trends Across Matrix Sizes.

- **Small Matrix Peak:** For $N = 512$, the speedups are at their maximum because the entire input, kernel, and output matrices fit within the 1.25 MiB L2 cache. Here, the bottleneck is purely the CPU's arithmetic throughput.
- **Large Matrix Stabilization:** For $N \geq 2048$, the speedup for 32-wide SIMD stabilizes at approximately **7.5x to 8x**. At this scale, the working set exceeds the L2 cache, and the performance becomes partially **Memory Bound**.
- **Tiling Observations:** Interestingly, the "SIMD + Tiling" (Gray bar) performs exceptionally well at small sizes but sees a sharp decline for the largest matrix ($N = 16384$, $\sim$4.5x). This suggests that for very large matrices on modern Intel hardware, the overhead of managing tile loops may interfere with the hardware prefetcher's ability to stream data effectively compared to the simpler linear access of the 32-wide SIMD implementation.

4.3.4 Conclusion for Task 1

The 32-wide SIMD implementation represents the "architectural sweet spot" for the Tiger Lake platform. By balancing the number of accumulators to match the hardware's FMA latency and port count, I achieved a nearly 2x improvement over the base SIMD version at small scales and a robust, consistent speedup at large scales. This progression proves that

truly optimized code must consider both the vector width and the pipeline depth of the underlying hardware.